

Buna ziua stimati membri ai comisiei, ma numesc... si va voi vorbi despre un protocol...Schnorr

Slide 2 - Cuprins

În introducere, voi expune tema și scopurile lucrării

Apoi tehnologiile utilizate.

Arhitectura generală a sistemului

Date introductive despre schema de identificare Schnorr interactivă

Fluxurile de înregistrare și autentificare alături de Proprietățile acestora

Iar în final avem rezultatele și concluziile

Slide 3 - Introducere

Tema pornește de la vulnerabilitățile specifice aplicațiilor web care folosesc un protocol de autentificare OAuth unde se transmite un secret direct fără criptare către serverul de autentificare. Deși protocolul HTTPS reduce riscul interceptării, acesta nu elimină vulnerabilități precum breșele bazelor de date, replay sau atacurile de tip Store Now, Decrypt Later.

Astfel, soluția propusă constă în proiectarea unui protocol de autentificare în care parola nu este transmisă niciodată către server-ul de autentificare iar prin integrarea schemei de identificare Schnorr am putea valorifica demonstrațiile cu cunoștințe zero pentru a elimina și evita câteva vulnerabilități.

Din moment ce protocolul e destinat aplicațiilor web, s-a dezvoltat și un formular standard de autentificare pe care îl poate integra o astfel de aplicație.

Slide 4 - Tehnologii

Pentru dezvoltarea server-ului s-a ales Python cu Flask datorită flexibilității și suportului cu o multitudine de biblioteci JavaScript, HTML și CSS au fost utilizate pe partea de client, deoarece oferă compatibilitate cu majoritatea framework-urilor folosite în aplicații web.

Slide 5 - Arhitectură

Arhitectura este pe trei niveluri.

Nivelul de prezentare cuprinde dispozitivul sau interfața web prin care utilizatorul introduce sau deține credențialele și unde este inițiată procedura de autentificare

Nivelul de aplicație, gestionează logica de verificare și autentificare a user-ului, managementul sesiunii și emiterea token-ului

Nivelul de date conține bazele de date pentru stocarea asocierii dintre user, secretul public și resursele protejate

Slide 6 - Algoritmul Schnorr

Schema Schnorr interactivă se bazează pe dificultatea problemei logaritmului discret și e folosită frecvent în domenii precum IoT, blockchain sau alte aplicații zkp.

Algoritmul începe prin a alege un număr prim P , un factor prim Q și un generator astfel încât $G \bmod P = 1$, restul parametrilor sunt din spațiul lui Q

În faza de inițializare, se generează o cheie publică y de către Prover și se trimite la Verifier

Ulterior, în sesiunea de identificare, proverul trimite un angajament către verifier,

Acesta din urmă îi întoarce o provocare aleatorie c din spațiul $\mathbb{Z}_q^*$,

În final, proverul folosește provocarea pentru a construi o soluție care mai apoi e folosită de Verifier pentru a vedea dacă se potrivește în egalitatea finală.

Slide 7 - Fluxul de înregistrare

Astfel, în implementare, protocolul începe cu înregistrarea unde userul introduce credențialele, aplicația obține parametrii globali de la server-ul de autentificare și derivează x folosind parola pentru cheia publică y trimisă la server care pe urmă își face verificările propuse și salvează legătura dintre y și utilizator.

(DETALII DESPRE verificare payload, apartenența la subgroup, verificare dacă e înregistrat deja)

Slide 8 - Autentificare (faza commit)

La autentificare, după introducerea credențialelor și schimbul de parametri, aplicația client generează un număr aleator r , calculează angajamentul cu el și îl transmite împreună cu id-ul clientului.

Serverul validează payload-ul, creează o sesiune temporară din care e făcută provocarea și o returnează împreună cu session id.

(DETALII DESPRE hash e shake-256)

Slide 9 - Autentificare (faza verify)

Mai departe, în faza de verify se calculează soluția folosind parametrul derivat, și îl transmite ca răspuns la provocare. Se face egalitatea finală și dacă e corectă, atunci se emite un token

Slide 10 - Consum token

Pe care, aplicația îl poate folosi ca să acceseze resurse protejate de la serverul de resurse.

Slide 11 - Proprietăți de securitate

În urma implementării putem vedea că serverul nu stochează niciun secret, reținând doar cheia publică y , ceea ce reduce breșele de date, la problema logaritmului discret

Valorile t , c și s sunt efemere, prevenind atacurile de tip replay și prin faptul că r și x nu sunt transmise, face interceptarea nula și reduce reușita unor decriptări ulterioare

Slide 12 - Validare și testare

Validarea acoperă 49 de teste, toate cu rată de succes 100%, unde

5 teste pozitive verifică scenarii de baza/happy paths de autentificare/inregistrare, absența parolei hashuite/criptate în baza de date, funcționarea concurența de utilizatori;

5 teste negative care acoperă autentificarea cu parolă greșită, expirarea sesiunii, comituri duplicate și înregistrarea duplicată;

31 de corner case-uri care testează valori în afara Z_Q , valori triviale pentru y și t , tipuri de date neconforme precum float, string sau null, și absența antet

8 teste de securitate care acopera erori off-by-one, și încercări de autentificări cu cheia publică furată.

Din simularile de atacuri observăm că

Din 10k de cereri nu au fost coliziuni pe c

injectarea de parametri slabi cu $P=23$, $Q=11$, $G=4$ respinsă prin HTTP 422 pentru subgrup invalid;

replay cross-sesiune ajunge să fie respins datorită valorilor efemere

un brute force devine nefezabil după o cheie de 64 de biți

Slide 13 - Evaluarea performanței

Pentru implementare s-a folosit Un P de 2048 respectând recomandările NIST

De aceea putem observa că operațiile ce includ exponentiere modulară sunt cele cu cel mai mare factor de timp, operația de validare având cea mai mare latență deoarece are 2 exponențieri

Slide 14 - Comparatie OAuth față de ZKP

Dacă e să comparăm cu un protocol standard OAuth 2.0, vedem cel mai mare minus al protocolului și anume latența

În primul tabel avem măsurători ale latenței pentru diferite implementări, prima ar fi schnorr cu calcule exponențiale având o medie de 64 ms, mai jos avem implementări de oauth2.0 simplu și cu pixy unde avem 2 ms pe autentificare și ultima e o bibliotecă folosită pentru oauth2.0 cu 0.8 ms per autentificare

În testele de debit concurrent realizate cu Locust pe 8 threaduri timp de 60 de secunde, pentru schnorr putem vedea 19 cereri pe sec pentru 10 utilizatori concurenți și 17 cereri pentru 100 utilizatori pe când oauth2.0 are între 440 și 500

Aceste comparații nu sunt perfect echivalente având în vedere calculele pe care le face schnorr și pe care le face protocoalele oauth

Slide 15 - Comparație cu protocoalele SRP și zk-SNARK

Așa ca s-a optat pentru compararea cu alte protocoale bazate pe zero knowledge proof cum ar fi SRP și zk-snark, Pentru o comparație fidelă a trebuit executat schnorr de 2 ori deoarece srp are o autentificare mutuală, adică o verificare din partea ambelor părți.

Știind asta, era de așteptat să vedem timpi foarte mari la calculele exponentiale și am trecut pe curbe eliptice

În final s-a adăugat și o variantă de schnorr pe curbe eliptice dintr-o bibliotecă ce folosește cod C și s-a ajuns la un total de 9.5 ms

Slide 16 - Concluzii

În concluzie implementarea demonstrează că schema Schnorr poate fi integrată cu succes într-un protocol de tip Open Authorization, din urma căruia un utilizator se poate autentifica și obține un token spre utilizare ulterioară anulând efectele unei breșe a bazei de date sau ale unei exfiltrări a datelor prin folosirea valorilor efemere și prin nepartajarea parolei către serverul de autentificare.

Ca limitări se remarcă latența crescută care ar putea fi redusă prin adăugarea curbelor eliptice și prin folosirea unor limbaje care permit un grad sporit de optimizare și verificarea la browser care poate fi îndeplinită printr-o combinație a protocoalelor OAuth2.0 cu schnorr sau un schnorr mutual

Slide 17 - Bibliografie selectivă

Acestea sunt câteva referințe din bibliografia utilizată.

Slide 18 - Mulțumesc

Vă mulțumesc pentru atenție!